

NiceGUI 测试中 User Fixture 详细解析

在 NiceGUI 自动化测试体系中，**User Fixture** 是基于 `pytest` 夹具（Fixture）机制封装的、用于快速初始化 `nicegui.testing.user_plugin.Page` 对象的复用组件，核心作用是简化测试用例中“用户交互插件”的初始化流程，避免重复编写应用启动、插件绑定代码，同时保证测试环境的隔离性与一致性。它并非 NiceGUI 内置的固定夹具，而是开发者基于 `testing.plugin` 和 `user_plugin` 封装的**工程级复用夹具**，是 NiceGUI 测试实践中最常用的基础夹具之一。

一、核心定位与设计初衷

在使用 `user_plugin` 编写测试用例时，每个用例都需要执行以下重复步骤：

1. 调用 `testing.run_app()` 启动 NiceGUI 应用；
2. 初始化 `user_plugin.Page` 对象并绑定 Playwright 页面；
3. 测试结束后销毁应用进程、关闭浏览器上下文。

User Fixture 针对这一痛点，将上述步骤封装为可复用的 `pytest` 夹具：

- 一次定义，所有测试用例可直接注入使用；
- 支持灵活配置夹具作用域（function/module/session），控制环境隔离粒度；
- 自动处理应用启动 / 销毁、端口分配、浏览器初始化，降低测试用例编写成本。

二、User Fixture 的基础定义与使用

1. 核心定义模板（`conftest.py`）

`pytest` 夹具需定义在 `conftest.py` 文件中（`pytest` 自动识别该文件），以下是最基础的 User Fixture 定义：

```
# conftest.py（测试目录根目录）
import pytest
from nicegui import testing
from nicegui.testing import user_plugin
from app import main # 导入你的 NiceGUI 应用入口函数

@pytest.fixture(scope="function") # 作用域：每个测试用例独立初始化
def user():
    """User Fixture: 初始化 user_plugin.Page 对象，绑定运行中的 NiceGUI 应用"""
    # 启动应用并获取 Playwright Page 对象
    with testing.run_app(
        app_function=main,
        port=None, # 随机端口，避免冲突
        browser="chromium", # 指定测试浏览器
        headless=True # 无头模式（CI/CD 推荐）
    ) as page:
        # 初始化 user_plugin.Page 对象（核心：绑定应用与交互插件）
        user_page = user_plugin.Page(page)
        yield user_page # 将对象注入测试用例
    # 夹具销毁时，with 上下文自动关闭应用、销毁浏览器
```

2. 测试用例中使用 User Fixture

```
# test_app.py
def test_button_click(user): # 直接注入 user fixture
    # 1. 定位按钮组件 (user_plugin 方法)
    button = user.get_by_text("提交")
    # 2. 模拟用户点击 (user_plugin 交互 API)
    user.click(button)
    # 3. 断言交互结果
    notification = user.get_by_text("点击成功")
    assert notification.exists()

def test_input_text(user): # 复用同一个 fixture 定义
    # 定位输入框并模拟输入
    input_box = user.get_by_label("用户名")
    user.type(input_box, "test_user")
    # 断言输入内容正确
    assert user.inner_text(input_box) == "test_user"
```

三、User Fixture 的关键配置

1. 作用域 (scope) 配置

Fixture 作用域决定了环境初始化 / 销毁的频率，核心可选值：

作用域	行为	适用场景
<code>function</code> (默认)	每个测试用例启动一次应用、创建一次 user 对象	需严格隔离的测试用例（如修改全局状态的用例）
<code>module</code>	整个测试模块（.py 文件）共享一个 user 对象	同模块内用例无状态冲突，追求测试效率
<code>session</code>	整个测试会话（所有用例）共享一个 user 对象	耗时较长的应用启动（如大型应用），需确保用例无状态污染

示例：module 作用域的用户 Fixture

```
@pytest.fixture(scope="module")
def user():
    with testing.run_app(main) as page:
        user_page = user_plugin.Page(page)
        yield user_page
    # 整个模块测试完成后才销毁应用
```

2. 自定义应用启动参数

可在 `testing.run_app()` 中配置更多参数，适配不同测试场景：

```

@pytest.fixture(scope="function")
def user():
    with testing.run_app(
        app_function=main,
        port=8080, # 固定端口（调试时方便）
        browser="firefox", # 切换测试浏览器
        headless=False, # 显示浏览器窗口（调试用）
        native=False, # 禁用 NiceGUI 原生窗口
        reload=False # 禁用热重载
    ) as page:
        user_page = user_plugin.Page(page)
        yield user_page

```

3. 前置操作与状态重置

可在 Fixture 中添加前置操作（如初始化测试数据、重置 UI 状态），确保每个用例的初始环境一致：

```

@pytest.fixture(scope="function")
def user():
    with testing.run_app(main) as page:
        user_page = user_plugin.Page(page)
        # 前置操作：清空所有输入框、关闭弹窗
        user_page.clear(user_page.get_by_label("用户名"))
        user_page.clear(user_page.get_by_label("密码"))
        # 等待页面加载完成
        user_page.wait_for(user_page.get_by_text("首页"))
        yield user_page

```

四、进阶用法：带配置的 User Fixture

1. 参数化 User Fixture

通过 `@pytest.mark.parametrize` 为 Fixture 传入参数，适配多场景测试：

```

# conftest.py
@pytest.fixture(scope="function")
def user(request):
    # 获取参数化的浏览器类型
    browser = request.param if hasattr(request, "param") else "chromium"
    with testing.run_app(main, browser=browser) as page:
        yield user_plugin.Page(page)

# test_app.py
import pytest

# 参数化：测试不同浏览器下的交互
@pytest.mark.parametrize("user", ["chromium", "firefox", "webkit"], indirect=True)
def test_cross_browser(user):
    button = user.get_by_text("提交")
    user.click(button)
    assert user.get_by_text("点击成功").exists()

```

2. 组合多个 Fixture (User + Screen)

在实际测试中，常需同时使用 `user_plugin` (交互) 和 `screen_plugin` (全局验证)，可封装组合夹具：

```
# conftest.py
@pytest.fixture(scope="function")
def test_env():
    """组合夹具：同时返回 user 和 screen 对象"""
    with testing.run_app(main) as page:
        user = user_plugin.Page(page)
        screen = screen_plugin.Screen(page)
        yield {
            "user": user,
            "screen": screen
        }

# test_app.py
def test_combined_interaction(test_env):
    # 1. user 模拟输入
    test_env["user"].type(test_env["user"].get_by_label("用户名"), "test")
    # 2. screen 全局验证
    test_env["screen"].assert_text("test")
    # 3. user 模拟点击
    test_env["user"].click(test_env["user"].get_by_text("提交"))
    # 4. screen 等待结果
    test_env["screen"].wait_for_text("提交成功")
```

3. 异步版本的 User Fixture

针对异步测试用例 (`pytest.mark.asyncio`)，需定义异步版 User Fixture：

```
# conftest.py
import pytest
import asyncio
from nicegui import testing
from nicegui.testing import user_plugin

@pytest.fixture(scope="function")
async def async_user():
    """异步 User Fixture"""
    async with testing.async_run_app(main) as page:
        user_page = user_plugin.Page(page) # user_plugin 兼容异步页面
        yield user_page

# test_app.py
@pytest.mark.asyncio
async def test_async_click(async_user):
    button = async_user.get_by_text("提交")
    await async_user.click(button) # 异步调用交互方法
    notification = async_user.get_by_text("点击成功")
    assert await notification.is_visible()
```

五、关键注意事项

1. 环境隔离性

- 若 Fixture 作用域为 `module/session`，需确保测试用例之间无状态污染（如避免修改全局 `ui.state` 后不重置）；
- 随机端口（`port=None`）是默认推荐配置，避免多用例并行时端口冲突。

2. Fixture 销毁逻辑

- 依赖 `testing.run_app()` 的上下文管理器，夹具销毁时会自动关闭应用、销毁浏览器上下文，无需手动调用 `page.close()`；
- 若在 Fixture 中手动创建了资源（如测试数据、临时文件），需在 `yield` 后添加清理逻辑。

3. 调试技巧

- 调试时将 `headless=False`，可直观看到浏览器中的交互过程；
- 结合 `pytest -s` 运行测试，输出 Playwright 日志，定位交互失败原因。

4. CI/CD 适配

- CI 环境中需确保安装 Playwright 系统依赖（`playwright install-deps`）；
- 建议使用 `session` 作用域减少应用启动次数，提升 CI 测试效率。

六、User Fixture 与原生 user_plugin 的对比

特性	User Fixture	直接使用 user_plugin
代码复用	一次定义，所有用例复用	每个用例需重复编写启动 / 初始化代码
环境隔离	可通过 scope 灵活控制	需手动管理隔离（如每次启动新应用）
配置复杂度	集中配置，便于统一管理	分散配置，易出现不一致
学习成本	需掌握 pytest 夹具基础	仅需掌握 user_plugin API
适用场景	多测试用例、工程化测试	单测试用例、临时调试